

Epigenetic Language Models: Lifelong Learning by Intercalating Knowledge into Structural Zeros

Free-Slot Adaptation on a real 8B ternary MoE

Parallax / Chutes.ai

October 9, 2026

AI-assisted draft. Revised through repeated adversarial review; experiments run and checked by the authors. Results are from one real 8B ternary MoE on one 40-fact task, single seed (plus a supporting synthetic isolated-expert probe). The ternary Bop repeat uses the same seed, facts and protocol; it does not establish robustness across seeds. The multi-session epigenesis experiment (EP-1) is pre-registered, in progress and untested; the personal-model use case (§8) is an *untested proposal*. Code: github.com/chutesai/epigenesis.

Abstract

An epigenetic language model keeps its base weights as a frozen genome and writes acquired knowledge into a separate sparse layer occupying exact zeros. Removing that layer restores the base bit-for-bit. We test Free-Slot Adaptation (FSA) on a real 8B ternary mixture-of-experts model. On one 40-fact task, the reference ternary Bop run recalls 80/80 held-out items from 812,420 filled slots, adding no parameters and preserving the packed format, with bit-exact revoke: in every run, patch-off logits are bitwise equal to the base on one 512-token held-out window, and test recall returns to 0/80. The patch costs 0.9 MB entropy-coded or 2.7 MB indexed, including positions: about 16–49 times smaller than a 44 MB bf16 LoRA adapter. General-text damage is +0.027 nats/token; the penalized LoRA baseline recalls 78/80 at damage indistinguishable from zero. Recall means greedy agreement on every answer token under teacher forcing, without testing termination or extra generated text. A two-item difference on one seed is not a significant recall win; LoRA remains the lower-damage method, and FSA does not beat it overall. Multi-session concept learning remains untested; the pre-registered EP-1 experiment is in progress.

1 Introduction

We use an epigenetic analogy: the base weights are a frozen genome, while acquired knowledge lives in a separate sparse layer that can be removed to recover the base bit-for-bit. Extreme quantization already leaves many weights at exact zero. Can those existing slots carry new knowledge without adding parameters?

Free-Slot Adaptation (FSA). In the pair8 ternary format, at most two of every four aligned pairs are active. Let $\mathcal{S}_0$ be the base’s nonzero support and $\mathcal{Z}_0$ its zeros. Freeze the base and all shared parameters, then train a disjoint slice $\mathcal{R}_d \subseteq \mathcal{Z}_0$ inside already-active pairs. Store its positions and signs as a removable patch. Enabling the patch uses $\mathcal{S}_0 \cup \mathcal{R}_d$; disabling it uses $\mathcal{S}_0$ alone. This is Mallya and Lazebnik [2018]’s mechanism, with the reserve supplied by the compression format rather than a pruning pass.

What we claim. On a real 8B ternary MoE, ternary Bop run 1 recalls 80/80 held-out items from 812,420 filled slots with bit-exact revoke and no new parameters. Its position-inclusive patch is 0.9 MB entropy-coded or 2.7 MB indexed, about 16–49 times smaller than the 44 MB bf16 LoRA baseline, but costs +0.027 nats/token general-text damage. Penalized LoRA recalls 78/80 at damage indistinguishable from zero. Two items on one seed do not establish a significant recall win. FSA does not beat LoRA overall; restoration when the patch is removed does not preserve behavior while it is enabled. Multi-session concept learning and the personal-model application remain untested.

Contributions. (1) Quantization zeros as a removable adaptation reserve, with accounting for positions, values, scales, and inference format. (2) Real-model results comparing ternary Bop, learn-then-prune, fp4, and LoRA on recall, damage, and storage. (3) A measured KL floor under random perturbations, with routing near-ties as a possible explanation rather than an established cause. (4) A patch-revocation contract and a pre-registered multi-session concept experiment (§8).

2 Background and related work

Mask-based continual learning. PackNet [Mallya and Lazebnik, 2018] freezes kept weights and trains freed ones, recovering a task by its mask; Piggyback [Mallya et al., 2018] *learns masks over fixed weights* (it does not refill zeros); HAT [Serra et al., 2018] uses hard task-attention and reports *reduced*, not zero, forgetting; Supermasks [Zhou et al., 2019] make the mask itself the trainable object; Progressive Networks [Rusu et al., 2016] add and could drop columns. FSA is PackNet with the free support supplied by the quantization format. **Quantized continual learning.** Ada-QPackNet [Pietroń et al., 2023] and TinySubNets [Pietroń et al., 2025] already combine PackNet-style capacity reuse with quantization; SPDF [Thangarasa et al., 2023] reopens zeroed pretrained weights for dense fine-tuning. These sharply narrow our novelty; our distinct element is a reserve *pre-addressed by the deployed format* and composed by union. **Parameter-efficient and continual adapters.** LoRA [Hu et al., 2022], QLoRA [Detrmers et al., 2023], QA-LoRA [Xu et al., 2024]; Biderman et al. [2024] find LoRA learns and forgets less than full fine-tuning (a possible rank explanation, not a theorem); O-LoRA [Wang et al., 2023] does continual learning in orthogonal LoRA subspaces. **Sparse / composable adaptation.** Diff pruning [Guo et al., 2021] and composable sparse fine-tuning [Ansell et al., 2022]; union of *disjoint* sparse supports is the easy case of the latter. **Editing, memory, unlearning.** MEMIT [Meng et al., 2023] edits facts in place and can override; FSA adds paths that can suppress or correct outputs while enabled but cannot edit a base weight; GRACE [Hartvigsen et al., 2023] and WISE [Wang et al., 2024] keep deletable, routed side-memory for sequential edits while freezing the base; SISA [Bourtole et al., 2021] is exact unlearning by shard isolation; TOFU [Maini et al., 2024] benchmarks fictitious-fact forgetting. **Personalization and retrieval.** OPPU [Tan et al., 2024] attaches a per-user PEFT module (removed by user identity, not per record); LaMP [Salemi et al., 2024] is a personalization benchmark with retrieval baselines; RAG [Lewis et al., 2020] and kNN-LM [Khandelwal et al., 2020] store facts externally and delete them by dropping a record—the baseline any “delete a fact” claim must beat. **Low-bit models.** BitNet b1.58 [Ma et al., 2024, 2025] and LittleBit [Lee et al., 2025]. Bop [Helwegen et al., 2019] optimizes binary weights without latent real weights; we extend its gradient-memory rule to ternary birth and death (§7).

3 Method

3.1 Format and reserve

In pair8, weights form aligned blocks of eight (four pairs); each weight is ternary times a scale, and at most two pairs per block are non-zero. A block thus has 417 reachable states ($1 + 4 \cdot 8 + \binom{4}{2} 8^2$), i.e. $\log_2 417/8 \approx 1.088$ bits/weight asymptotically (1.125 at a fixed 9 bits/block), and is $\geq 50\%$ zero *by construction* (ternary alone guarantees no such fraction). **Writable reserve.** Zeros inside already-active pairs can be filled while preserving pair occupancy and the codebook. At two active pairs, opening a third pair is illegal, but filling a zero member of an active pair is legal. Thus the writable in-codebook reserve is a subset of the full zero support, not necessarily empty in a full-occupancy block. With pair occupancy frozen, opening new pairs requires a separate sidecar patch outside the base codebook. Filled positions and signs must still be stored for revocation; the joint rate of 1.088 bits/weight is not the marginal price of a filled slot.

3.2 Adaptation and exact revoke

Train only the free-slot patch ΔW , with support $\mathcal{R}_d \cap \mathcal{S}_0 = \emptyset$, while the base remains live. Base weights, per-row scales, pair occupancy, router, and norms are frozen. A legal ternary writer fills free slots with $\pm\alpha$, where α is the frozen row scale. The enabled model uses $W + \Delta W$; zeroing the filled slots returns every patched tensor bit-identical to W , since the patch never overwrites base support or shared state. In every run, patch-off logits were bitwise equal to the base on one 512-token held-out window, and test recall returned to 0/80.

This restores the base, without guaranteeing its behavior while the patch is on. It does not delete individual facts within a patch. If later sessions train with an earlier patch enabled, zeroing that session’s slots alone need not remove its influence on retained patches or training state (§8).

3.3 Honest accounting

Plain ternary FSA adds **zero new logical parameters**: the element count is unchanged. It fills only zeros inside active pairs, so a merged patch uses the same packed tensors and adds no tensors or inference compute. This claim depends on preserving pair occupancy and the serving format; opening new pairs or using a separate patch can add work. The packed serving path was not benchmarked (§4). Storage is not free: revocation needs positions and signs in a stored diff, or the original pack. The learned-block-scale arm adds parameters and storage and is outside these packed-format claims. A full per-domain mask would cost one bit per weight; sparse position coding avoids that

cost for sparse patches. Frozen row scales constrain each ternary slot to $\{-\alpha, 0, +\alpha\}$, a capacity restriction. Actual position-inclusive patch sizes are reported in §6; hypothetical trit-only accounting is insufficient.

4 Real-model setup

Lambda is an 8B-parameter ternary MoE with 128 routed experts per MoE layer, 12 active per token, model width 1152, and a recurrent sequence-mixing state between layers. Each routed expert has a 2304×384 up projection, squared ReLU, and a 384×2304 down projection, stored as `pair8` trits times frozen per-row scales. We patch layers 14–15, all 128 experts, both projections: a broad footprint with 66,807,179 free slots inside already-active pairs. Base support, scales, pair occupancy, router, and all other parameters remain frozen. Plain ternary births take value $\pm\alpha$ for row scale α . The block-scale arm keeps ternary signs but multiplies them by learned magnitudes, one per row and 16 input columns, initialized at 0.5α . Its values leave $\{-\alpha, 0, +\alpha\}$; the packed-format and no-new-parameter claims do not cover this arm. The fp4 variant uses magnitudes $\{0, 0.125, 0.25, 0.375, 0.5, 0.75, 1, 1.5\}\alpha$ (the E2M1 grid scaled by 0.25α), with minimum nonzero magnitude 0.125α and cap 1.5α ; it leaves the ternary codebook. Evaluation decodes experts to bf16. Legal ternary patches could use the same packed kernel, but that serving path was not benchmarked.

The task has 40 synthetic facts about a fictional person (favorite cheese, boat’s name, house number). Training lasts 300 steps with answer-token-only loss. Each step samples three phrasings per fact from a pool of 12 templates (three plain sentences and nine augmented forms, including “Question:/Answer:” and “User:/Assistant:”). Two further phrasings and 16×512 general-text windows provide validation monitoring and checkpoint selection. Test recall is scored under teacher forcing: given the prompt, the model’s greedy (argmax) prediction must match every answer token; termination and extra generated text are not tested. Recall is measured on two phrasings never used in training or validation: “Q: What is X’s attr? A:” and “A little-known fact about X: her attr is”. The Q/A form is held out verbatim but resembles trained question/answer formats. There are 80 test items; one item is 1.25 percentage points, and the unpatched model gets 0/80. Damage is mean per-token Δ NLL relative to the base on 40 disjoint 512-token WikiText windows (about 20k tokens), with paired standard error (SE) over windows. The unpatched model’s NLL is 2.890 on a separate small diagnostic pool of five 512-token held-out windows, which also provides the reference for the chaos-floor NLL ranges. All results use one seed, with two runs of the same $\tau = 0.7$ ternary Bop configuration; Table 2 reports step 300.

5 A chaos floor for KL

Random, untrained ternary flips produce a KL floor while NLL on the separate five-window diagnostic pool stays within about 0.01 nats of the unpatched model. For the down-projection flips in Table 1 and Figure 1, the range is about -0.010 to $+0.003$ relative to the diagnostic-pool base NLL of 2.890; across all flip settings tested, it is about -0.010 to $+0.005$. A single layer-15 zero in one expert’s down projection illustrates the bf16 threshold: changes at most 0.01α round away; a full trit changes about 6% of top-1 tokens. Two different single flips disagree with each other by KL 0.027, slightly more than one flip differs from the base (0.024), and the divergence is roughly flat across sequence position. These observations suggest a different but equally good trajectory after a change exceeds bf16 resolution. Near-ties in discrete MoE routing, propagated through recurrent state, are a plausible driver, not a proven explanation.

Random flips	Layer 2 KL	Layer 15 KL	Layer 28 KL
1	0.027	0.023	0.007
1,000	0.027	0.023	0.015
1,000,000	0.028	0.026	0.023

Single change	KL	Top-1 agreement
$\varepsilon \leq 0.01$	0	1.000
$\varepsilon = 0.1$	0.0105	0.978
$\varepsilon = 1$	0.0238	0.939

Table 1: Untrained perturbations (single seed). Top: random free-slot flips in down projections. Bottom: one layer-15 weight changed by $\varepsilon\alpha$; small changes are absorbed by bf16 rounding.

For always-on edits the typical KL floor is about 0.02–0.027, with layer-dependent exceptions in Table 1. A KL

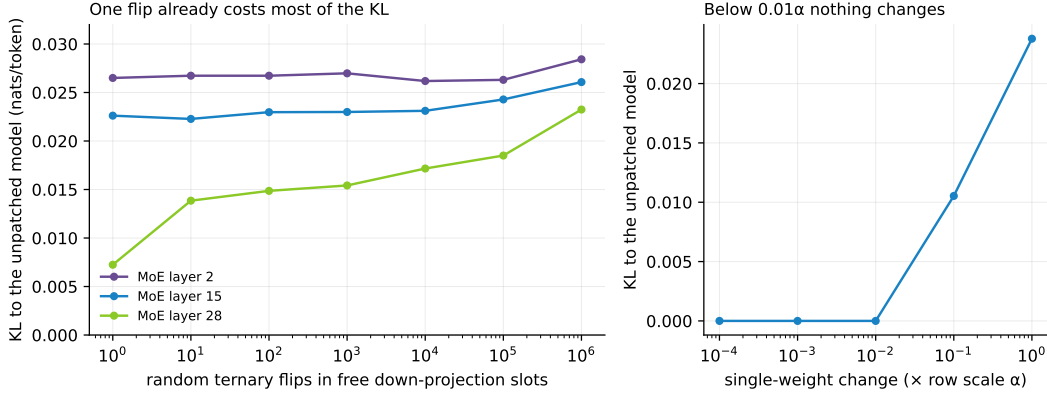


Figure 1: KL against the number of random ternary flips in free down-projection slots at layers 2, 15, and 28 (left), and against a single-weight change relative to its row scale (right). Unlike layers 2 and 15, layer 28 has a lower KL of 0.007 for a single flip, rising to 0.023 at one million flips. NLL on the separate diagnostic pool of five 512-token held-out windows remains within about 0.01 nats of its base NLL of 2.890 (range about -0.010 to $+0.003$) for these down-projection flips; across all flip settings tested, the range is about -0.010 to $+0.005$.

penalty mostly fights this floor: increasing its weight from 1 to 30 left KL near 0.06 while recall fell from 48% to 20%. We therefore measure damage with paired ΔNLL , interpret KL relative to this floor, and use a local output penalty for preservation (§7).

6 Results

6.1 Recall and damage

Method	Test recall	$\Delta\text{NLL} \pm \text{SE}$	Nonzero entries
Ternary Bop, $\tau = 0.7$ (run 1)	100% (80/80)	$+0.0266 \pm 0.0035$	812,420
Ternary Bop, $\tau = 0.7$ (run 2, repeat)	100% (80/80)	$+0.0237 \pm 0.0036$	669,705
Ternary Bop, $\tau = 0.9$	100% (80/80)	$+0.0314 \pm 0.0033$	588,673
Ternary learn-then-prune, decay 0.5	88.75% (71/80)	$+0.0129 \pm 0.0028$	523,229
Ternary learn-then-prune, decay 0.3	91.25% (73/80)	$+0.0250 \pm 0.0032$	542,205
Ternary learn-then-prune, decay 0.1	93.75% (75/80)	$+0.0667 \pm 0.0035$	1,857,504
Signs + learned block scales (16), decay 0.3	87.5% (70/80)	$+0.0282 \pm 0.0031$	1,125,769
Fp4 + local penalty $\lambda = 10$, decay 0.1	96.25% (77/80)	$+0.0050 \pm 0.0015$	21,265,606
LoRA $r = 16$ + local penalty (baseline), $\lambda = 100$	97.5% (78/80)	$+0.0005 \pm 0.0018$	22,020,096
LoRA $r = 16$, unregularized (baseline)	97.5% (78/80)	$+0.936 \pm 0.030$	22,020,096

Table 2: Single-seed final checkpoints: 40 facts, layers 14–15, all experts, up and down, answer-only loss, 12 formats. In every run, switching the patch or adapter off gives bitwise-equal logits to the unpatched model on one 512-token held-out window and returns test recall to 0/80. FSA counts slots; LoRA counts fp32 parameters as trained.

The two $\tau = 0.7$ ternary Bop runs use the same seed, facts and protocol. Their spread (812,420 vs 669,705 slots; damage $+0.0266$ vs $+0.0237$ nats/token, a damage difference within one SE) is variation between repeated executions, plausibly from nondeterministic GPU arithmetic; it is not evidence of robustness across seeds. The repeat’s configuration also listed weight decay, but decay is not part of the ternary Bop update (there are no real-valued weights to decay), so it is the same configuration as run 1.

Checkpoint selection uses a family of validation-recall floors; each floor keeps the checkpoint with the lowest

validation ΔNLL among those meeting it. For LoRA + penalty, floors 0.3–0.7 select step 100, giving 70%/+0.0061 test recall/damage; the 0.8 floor selects step 150, giving 95%/+0.0029. The selected ternary Bop $\tau = 0.7$ run-1 checkpoint at step 250 gives 100%/+0.0268; run 2 selects step 100 at every floor, giving 100%/+0.0247. Bop $\tau = 0.9$ at step 50 gives 100%/+0.0318, and learn-then-prune at step 250 gives 90%/+0.0282 for decay 0.3. Decay 0.5 selects step 300, giving 88.75%/+0.0129. Under these selections penalized LoRA stays below every ternary writer in damage; fp4’s step-300 checkpoint (+0.0050, 96.25%) is slightly below LoRA’s floor-0.3–0.7 checkpoint (+0.0061, 70%), within about one combined SE. At their selected checkpoints, both Bop $\tau = 0.7$ runs and learn-then-prune decay 0.3 differ in damage by less than one combined (quadrature) SE; learn-then-prune decay 0.1 is clearly worse (+0.067).

Bop’s 80/80 against LoRA’s 78/80 matches, perhaps slightly exceeds, recall: two items on one seed do not establish a significant win. LoRA with the local penalty has damage indistinguishable from zero (+0.0005 $\pm$ 0.0018); ternary Bop run 1 costs about +0.027 nats/token (about 2.7% perplexity) (0.0266 vs 0.00048), though LoRA’s interval includes zero. Fp4 is closest to LoRA on both axes (96.25%, +0.005), but leaves the ternary format and uses a much larger patch. Unregularized LoRA is far more damaging (+0.94) than any FSA run in Table 2. Stronger learn-then-prune decay trades recall for damage: decay 0.5 gives 71/80 (+0.0129) versus 73/80 (+0.0250) at decay 0.3. This is the lowest-damage plain ternary result, still about 27 times LoRA’s point estimate, with nine missed items; LoRA’s interval includes zero. Plain ternary writers achieve their moderate damage without a penalty term; the ternary magnitude pin bounds how far a slot can move.

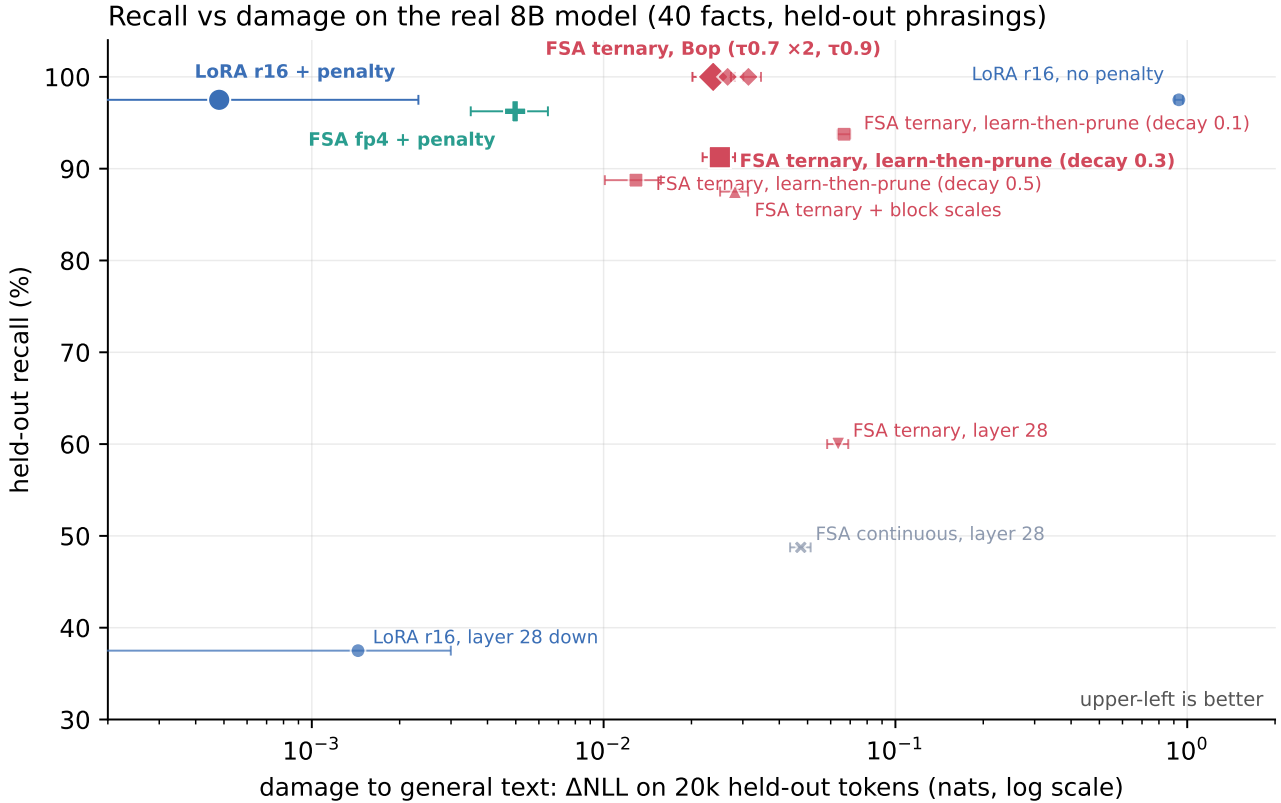


Figure 2: Held-out recall against general-text ΔNLL (log horizontal axis, paired-SE bars). Error bars extending below zero are truncated on the log axis; LoRA + penalty’s interval and the late-layer LoRA interval (+0.0014 $\pm$ 0.0016) both include zero. Protocol as in Table 2: answer-only loss, 12 formats (3 plain, 9 augmented), recall on two held-out phrasings (one Q/A), ΔNLL on 40 disjoint 512-token WikiText windows. Besides Table 2, the two late-layer FSA arms use layer 28, down only, with a 2M-slot budget: gain-minus-cost ternary selection gives 60%/+0.064 and continuous bf16 slots 48.75%/+0.047. Same-placement LoRA (layer 28, down only, $r = 16$ + penalty) has 5,505,024 parameters and no slot budget, and gives 37.5%/+0.0014. All use one seed; the Bop group includes two $\tau = 0.7$ runs and one $\tau = 0.9$ run. The learn-then-prune points include decay 0.5, 0.3, and 0.1.

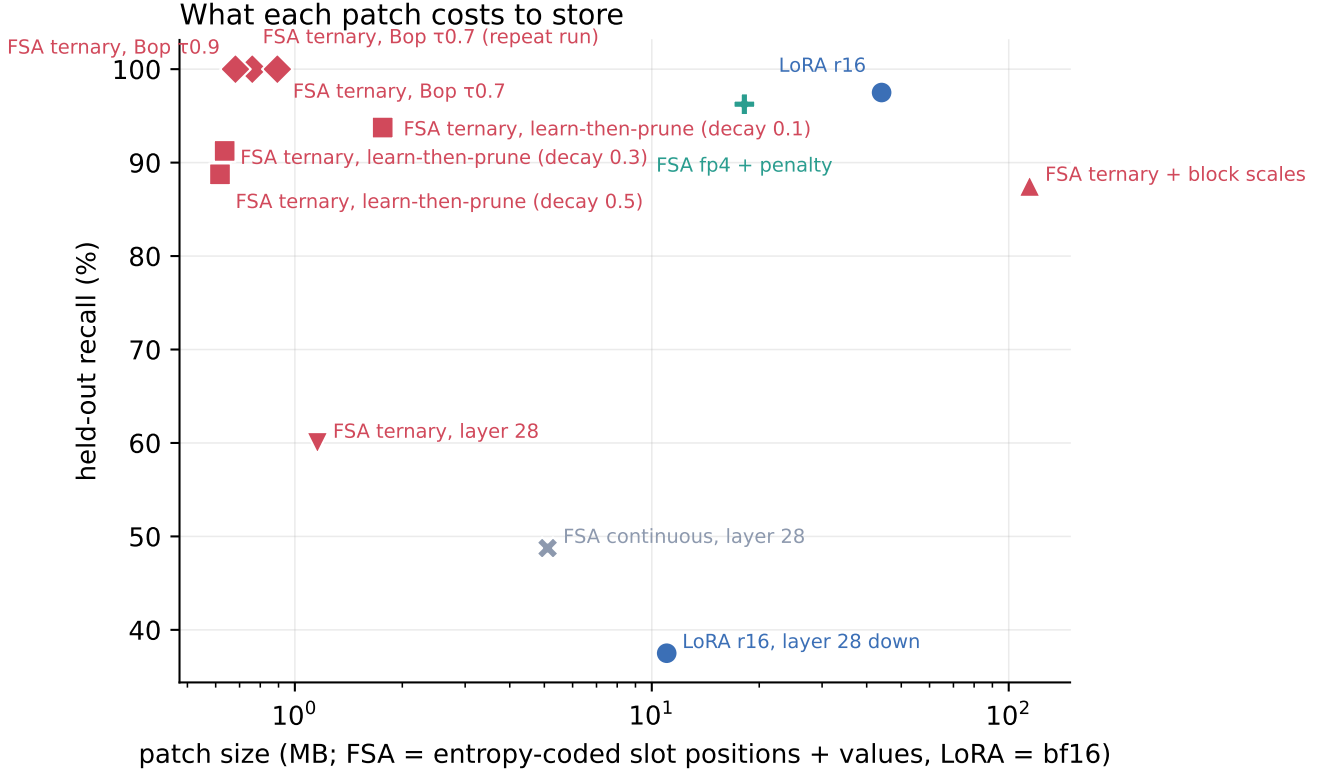


Figure 3: Recall against patch size. Sizes count slot positions, not only values: FSA is entropy-coded positions plus values, with positions coded over each run’s own free-slot pool: 66.8M slots at layers 14–15 and 16.8M for the layer 28 down-only arms. LoRA counts bf16 parameters. The plot includes the learned fp32 block scales, bringing that run to about 114 MB, larger than the adapter. The $\tau = 0.7$ repeat is labelled separately; learn-then-prune includes decay 0.5, 0.3, and 0.1.

6.2 Storage

Patch	Entropy-coded	Indexed
Ternary Bop $\tau = 0.7$ (run 1), 812,420 slots	0.9 MB	2.7 MB
Ternary Bop $\tau = 0.7$ (run 2, repeat), 669,705 slots	0.8 MB	2.3 MB
Ternary Bop $\tau = 0.9$, 588,673 slots	0.7 MB	2.0 MB
Learn-then-prune, decay 0.5, 523,229 slots	0.6 MB	1.8 MB
Learn-then-prune, decay 0.3, 542,205 slots	0.6 MB	1.8 MB
Learn-then-prune, decay 0.1, 1,857,504 slots	1.8 MB	6.3 MB
Block-scale run, 1,125,769 slots	1.2 MB + block scales	3.8 MB + block scales
Fp4 + penalty, 21.3M slots	18 MB	80 MB
LoRA $r = 16$ + local penalty (baseline)	22,020,096 params; 44 MB bf16 / 88 MB fp32	

Table 3: Position-inclusive storage estimates. Ternary indexed storage uses a 26-bit index per slot plus its sign; fp4 indexed storage uses a 26-bit index plus a 4-bit value. Block scales (one per row and 16 input columns) add about 113 MB in fp32 as implemented, larger than the adapter. The LoRA baseline has 22,020,096 parameters: 44 MB in bf16, 88 MB in fp32 as trained.

6.3 Honest reading

Ternary Bop $\tau = 0.7$ run 1 is about 16–49 times smaller than the 44 MB bf16 adapter using unrounded estimates, depending on position coding (Table 3). Across the plain ternary patches the ratios range about 7–71 times, with learn-then-prune decay 0.1 giving the lowest ratio and decay 0.5 the highest; the $\tau = 0.7$ repeat is about 19–58 times

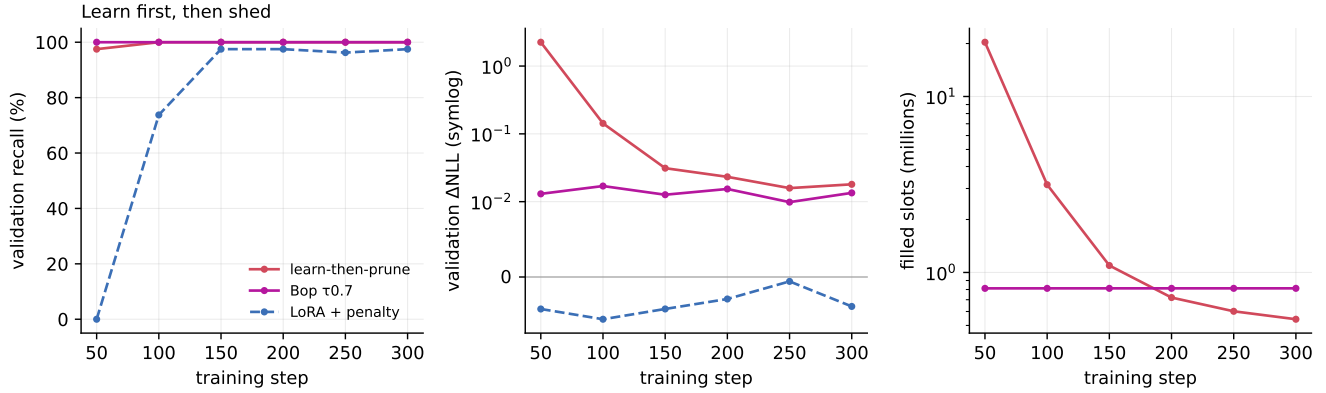


Figure 4: Validation recall, validation ΔNLL (symmetric log scale), and filled slots (millions, log scale) across training. The recall and damage panels show learn-then-prune, ternary Bop $\tau = 0.7$ run 1, and regularized LoRA; learn-then-prune uses decay 0.3. The filled-slot panel shows only those two FSA writers; the new repeat and decay 0.5 trajectories are not plotted. Bop’s filled-slot count is set by its threshold early and then stays flat, while learn-then-prune fills many slots first and sheds them over training. Validation trajectories differ from the held-out test endpoints in Table 2.

smaller; fp4 at 18–80 MB is not a small patch. Merged plain ternary FSA adds no parameters or tensors and, by construction, no arithmetic in a packed kernel (not benchmarked; we evaluated decoded bf16 weights), retaining legal packed values; revocation requires the diff or original pack. Unmerged LoRA adds a low-rank matrix multiplication per patched expert; merged LoRA requires dense bf16 expert weights and forfeits the ternary packed format. On this fact task, regularized LoRA remains the lower-damage method; ternary FSA trades measurable damage for matching recall, a smaller patch, unchanged tensor shapes and format, and exact revoke. Multi-session concepts remain untested.

6.4 Isolated experts

This is a supporting weight-level mechanism study on isolated decoded experts, using a synthetic random-fact probe read out through a random readout head; both favor FSA. Accuracy here is not language-model answer recall. The ternary reserve learned about 500 facts per expert at about 100% accuracy and revoked bit-exactly, but realized only 0.03–0.07 knowledge bits per slot versus 1.1–1.4 for float slots. In a separate comparison at about 2,000 facts, the full ternary reserve (all structural zeros, requiring a sidecar outside the codebook) had 8–20 times lower weight-level interference than LoRA $r=16/r=64$ under the same drift penalty at matched knowledge. That metric is isolated output drift, distinct from end-to-end general-text damage; on the full model, LoRA with a penalty is better.

7 Writers

7.1 Ternary Bop

Bop is Helwegen et al.’s binary optimizer [Helwegen et al., 2019]. Our ternary extension adds the birth and death rules below and resets the gradient EMA after every change. Each free slot holds $q_j \in \{-1, 0, +1\}$ directly, with no latent real weight. For loss gradient g_j , maintain

$$m_j \leftarrow (1 - \gamma)m_j + \gamma g_j, \quad \gamma = 0.05.$$

If $q_j = 0$ and $|m_j| > \tau$, birth sets $q_j \leftarrow -\text{sign}(m_j)$. If $q_j \neq 0$, $|m_j| > \tau$, and $\text{sign}(m_j) = q_j$, death sets $q_j \leftarrow 0$: the gradient says the current sign hurts. Reset m_j to zero after each change. Thresholds are 0.7 or 0.9 times the RMS of the first answer-loss gradient over free slots, with no penalty or decay. The threshold controls flips: small gradients need not become full $\pm\alpha$ steps, unlike Adam’s per-coordinate normalization, which can turn persistent small gradients into full flips. The slots are filled early: 812,077 slots were filled at step 50 (812,420 at step 300) at $\tau = 0.7$. Higher τ uses fewer slots (589k) at slightly higher damage, so it is not a simple damage dial. The $\tau = 0.7$ repeat has 669,465 filled slots at step 50 and 669,705 at step 300; its revoke is also bit-exact, with test recall returning

to 0/80.

7.2 Learn-then-prune

Each free slot has an fp32 master w_j in row-scale units. Forward uses $q_j = \text{sign}(w_j)\mathbf{I}(|w_j| > 0.5)$, where $\mathbf{I}$ is the indicator; gradients pass straight through [Bengio et al., 2013]. AdamW [Loshchilov and Hutter, 2019], learning rate 0.02, applies decoupled weight decay 0.3 to masters. The model learns early: at step 50 validation recall is 97.5%, with 20.3M filled slots and validation $\Delta\text{NLL} +2.2$. Decay pulls weakly supported masters under threshold: 3.2M slots at step 100, 1.1M at 150, and 0.54M at 300. Validation damage falls to about +0.02 and validation recall stays 100%; test recall is separately 91.25%. Held-out damage fell from +0.0282 at step 250 to +0.0250 at step 300, while validation damage rose slightly (+0.016 $\rightarrow$ +0.018). With decay 0.5, filled slots fall from 14.18M at step 50 (98.75% validation recall) to 1.12M at step 100, 0.62M at 150, 0.50M at 200, and 0.45M at 250 (92.5% validation recall), then rise to 0.52M at 300 (97.5% validation recall). Test recall at step 300 is 88.75%; revoke is bit-exact. Whether longer pruning helps is untested.

7.3 Fp4 and the local output penalty

Four-bit float slots times α use AdamW decay 0.1 and add the general-text penalty

$$\lambda \frac{\sum_{l,t} \|x_t \Delta W_l^T\|^2}{\sum_{l,t} \|x_t W_l^T\|^2 + 10^{-12}}, \quad \lambda = 10,$$

where l ranges over patched linear projections, t over general-text tokens, x_t is the input to that projection, W_l is its frozen base weight, and ΔW_l is the patch. The denominator, base-weight output energy on the current inputs, is treated as a constant normalizer and is not differentiated. This smooth local penalty measures each patched projection’s direct output change rather than the change in final logits, so it is not dominated by the whole-model divergence behind the KL floor. Its inputs still come from the patched forward pass: earlier patched layers can change later inputs and routing. The best LoRA run uses the same penalty with $\lambda = 100$. For ternary slots it is ineffective: gradient is zero at zero, yet one birth costs a full $\alpha^2 \mathbb{E}[x^2]$; at the strength LoRA needs, it stops ternary learning entirely.

7.4 Ingredients that mattered

Answer-only loss avoids learning template boilerplate, which caused most early damage with whole-sentence loss. Twelve diverse formats, including Q/A, largely closed the validation-to-test gap (Bop and penalized LoRA test at their validation recall; learn-then-prune decay 0.3 remains 100% validation vs 91.25% test). An earlier protocol reached only 54%/56% held-out recall. The broad footprint mattered: 32 selected experts in one layer, down only, with a 500k-slot budget (350k–404k slots filled) starved capacity (19–29% recall). Damage uses the roughly 20k-token held-out pool, with checkpoint selection on a separate validation pool.

7.5 What did not work

KL penalties fight the chaos floor and cost recall. Ternary slots with smooth penalty and decay peak in recall, then are pruned away. Tiny fp4 slots in late layer 28, down only, with a 2M-slot budget and grid magnitudes starting at 0.03125α , fill about 2.0M slots but yield 0% recall at +0.0067 damage: the patch changed the model but learned none of the tested answers. Patch-only learned block scales are no better than plain ternary (87.5%/+0.028) and add substantial storage. Single-late-layer continuous bf16 slots give 48.75%/+0.047, versus same-placement LoRA’s 37.5%/+0.0014: a recall–damage trade, not an overall improvement over LoRA. Gain-minus-cost ternary slot selection at that placement gives 60%/+0.064.

8 Toward epigenesis: multi-session learning

The ternary reserve now carries facts on a real model (§4). A revocable domain patch is therefore demonstrated for this task: enable the slice for its domain; disable it to recover the base. The personal-model application remains an untested proposal, with a narrower contract:

Patch-revocation contract. Deleting an isolated patch exactly removes *that patch’s parameter contribution*. Equivalence to a model trained without its data additionally requires that no retained patch or training state depended on it. It does not delete individual facts (facts within a slice are superimposed), and it does not make information already in the base inaccessible.

Retrieval deletes individual records while leaving the base unchanged [Lewis et al., 2020, Khandelwal et al., 2020]; disabling a per-user adapter removes that user’s contribution [Tan et al., 2024]. The case for weights is skills, style, and policy that benefit from adaptation in every forward pass. Arbitrary facts remain a retrieval baseline’s natural task. The real ternary fact experiment offers smaller patches at greater damage, not a general personalization result.

The pre-registered EP-1 experiment is in progress and explicitly **untested**: five sessions with a synthetic user containing 40 policies, 100 recurrent facts (each in 3 of 5 sessions), and 100 one-off “lure” facts (20 per session), consolidating self-study into free ternary slots. An identifiability gate requires the in-context ceiling to beat the unpatched model by at least 10 concept accuracy points on development probes; otherwise nothing is scored. Success criteria fixed in advance require, after session 5, concept gain of at least 10 concept accuracy points over the unpatched model on held-out probes, lure recall no more than 5 accuracy points above it, and mean general-text ΔNLL at most $+0.01$ nats/token on a locked 40×512 WikiText panel, disjoint from the selection pool, with a paired 95% damage interval excluding $+0.05$. Final concept accuracy must retain at least 80% of the best earlier session’s accuracy, with no upward ratchet in lure recall (monotone non-decreasing with a net increase).

The pre-registered ternary arm (“ternary with block scales”) uses a 2M-slot ceiling and learned block scales initialized at 0.25α , so like the block-scale arm here it is outside the packed-format / no-new-parameter claims. A separate fp4 arm is also registered; the comparator is regularized LoRA at the same placement.

The pre-registered revoke criterion clears the patch, scales, optimizer and training state, replay, and session ledger. It requires every patched weight tensor to be bitwise equal to the base and format-legal, logits on 32 fixed prompts within 10^{-2} of the base, and concept/lure metrics back to the base within noise. Exact logit equality is reported separately; it was observed when switching patches off in the fact experiment (§4). “Match LoRA” means within 3 concept accuracy points and no more than $+0.005$ extra damage, with the lure constraint held and revoke passing; “beat LoRA” means at least 5 more concept accuracy points with damage no worse than LoRA. Per the pre-registration, one seed is a pilot; three seeds are needed to claim. Neither multi-session concept learning nor on-device training is established: optimizer state can be large even when a final patch is small. A broader evaluation should include local retrieval and per-user adapters on personalization benchmarks [Salemi et al., 2024], with unlearning measured [Bourtole et al., 2021, Maini et al., 2024].

9 Limitations

Damage gap. The headline plain ternary writers have $+0.013$ to $+0.031$ nats/token damage (the 80/80 ternary Bop runs: $+0.024$ to $+0.031$), versus approximately zero for penalized LoRA; learn-then-prune decay 0.1 has $+0.067$ damage. The lowest-damage plain ternary run, decay 0.5, misses nine items (71/80). Closing this gap is current work. Combining Bop with the local penalty is being tried, with no results yet; longer pruning also remains untested.

Evaluation scope. We have one task: 40 facts about one fictional person, 80 test items, and one seed, with a same-configuration ternary Bop repeat using the same seed, facts and protocol. Variation between repeated executions plausibly comes from nondeterministic GPU arithmetic; it is not evidence of robustness across seeds. The Q/A test form resembles training. Writer and regularization choices differ. The chaos-floor explanation is a hypothesis, and the roughly 20k-token damage pool cannot establish broad behavioral safety. Multi-session concept learning remains untested in EP-1.

Revoke scope. Removing the whole patch restores the base bit-for-bit. This does not guarantee preservation while the patch is on, deletion of individual facts, removal of information already in the base, or removal of an earlier session’s influence after later sessions trained on top of it. The frozen base can be augmented or suppressed through new paths but its existing weights cannot be refined.

Format and deployment. Plain ternary writers stay legal pair8, but evaluation decoded experts to bf16; packed-kernel serving was not benchmarked. Fp4 and learned-block-scale variants leave the original codebook. Optimizer state can be large even when the final patch is small, and on-device training is not established.

Privacy. Knowledge in weights can be elicited. A per-user model is the proposed deployment for the personal application; its safety is untested, and the patch mechanism supplies no privacy guarantee.

10 Conclusion

A frozen weight genome can acquire a removable sparse layer in its exact zeros. On a real 8B ternary MoE, ternary Bop run 1 recalls 80/80 held-out items from 812,420 filled slots with bit-exact revoke, legal packed values, and no new parameters. Including positions, its patch costs 0.9 MB entropy-coded or 2.7 MB indexed, about 16–49 times smaller than the 44 MB bf16 LoRA baseline, at $+0.027$ nats/token damage. Penalized LoRA recalls 78/80 at damage indistinguishable from zero. Two items on one seed are not a significant recall win, and FSA does not beat LoRA overall. Reducing damage remains current work. Multi-session concept learning and the personal-model application

are untested; the pre-registered EP-1 experiment is in progress.

References

- Alan Ansell, Edoardo Maria Ponti, Anna Korhonen, and Ivan Vulić. Composable sparse fine-tuning for cross-lingual transfer. In *ACL*, 2022.
- Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv:1308.3432*, 2013.
- Dan Biderman, Jacob Portes, et al. LoRA learns less and forgets less. *TMLR*, 2024. arXiv:2405.09673.
- Lucas Bourtole et al. Machine unlearning. *IEEE S&P*, 2021. arXiv:1912.03817.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *NeurIPS*, 2023. arXiv:2305.14314.
- Demi Guo, Alexander M. Rush, and Yoon Kim. Parameter-efficient transfer learning with diff pruning. *ACL*, 2021. arXiv:2012.07463.
- Thomas Hartvigsen et al. Aging with GRACE: Lifelong model editing with discrete key-value adaptors. *NeurIPS*, 2023. arXiv:2211.11031.
- Koen Helwegen, James Widdicombe, Lukas Geiger, Zechun Liu, Kwang-Ting Cheng, and Roeland Nusselder. Latent weights do not exist: Rethinking binarized neural network optimization. In *NeurIPS*, 2019. arXiv:1906.02107.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022. arXiv:2106.09685.
- Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. *ICLR*, 2020. arXiv:1911.00172.
- Banseoek Lee et al. LittleBit: Ultra low-bit quantization via latent factorization. *NeurIPS*, 2025. arXiv:2506.13771.
- Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 2020. arXiv:2005.11401.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019. arXiv:1711.05101.
- Shuming Ma, Hongyu Wang, et al. The era of 1-bit LLMs: All large language models are in 1.58 bits. *arXiv:2402.17764*, 2024.
- Shuming Ma et al. BitNet b1.58 2b4t technical report. *arXiv:2504.12285*, 2025.
- Pratyush Maini et al. TOFU: A task of fictitious unlearning for LLMs. *COLM*, 2024. arXiv:2401.06121.
- Arun Mallya and Svetlana Lazebnik. PackNet: Adding multiple tasks to a single network by iterative pruning. In *CVPR*, 2018. arXiv:1711.05769.
- Arun Mallya, Dillon Davis, and Svetlana Lazebnik. Piggyback: Adapting a single network to multiple tasks by learning to mask weights. In *ECCV*, 2018. arXiv:1801.06519.
- Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. *ICLR*, 2023. arXiv:2210.07229.
- Marcin Pietroni, Kamil Faber, Dominik Żurek, and Roberto Corizzo. TinySubNets: An efficient and low-capacity continual learning strategy. *AAAI*, 2025. arXiv:2412.10869.
- Marcin Pietroni et al. Ada-QPacknet – adaptive pruning with bit width reduction as an efficient continual learning method without forgetting. *ECAI*, 2023. arXiv:2308.07939.
- Andrei A. Rusu et al. Progressive neural networks. *arXiv:1606.04671*, 2016.
- Alireza Salemi et al. LaMP: When large language models meet personalization. *ACL*, 2024. arXiv:2304.11406.
- Joan Serra, Didac Suris, Marius Miron, and Alexandros Karatzoglou. Overcoming catastrophic forgetting with hard attention to the task. In *ICML*, 2018. arXiv:1801.01423.
- Zhaoxuan Tan et al. Democratizing large language models via personalized parameter-efficient fine-tuning. *EMNLP*, 2024. arXiv:2402.04401.
- Vithursan Thangarasa et al. SPDF: Sparse pre-training and dense fine-tuning for large language models. *UAI*, 2023.
- Peng Wang et al. WISE: Rethinking the knowledge memory for lifelong model editing of large language models. *NeurIPS*, 2024. arXiv:2405.14768.
- Xiao Wang et al. Orthogonal subspace learning for language model continual learning. *EMNLP Findings*, 2023. arXiv:2310.14152.
- Yuhui Xu et al. QA-LoRA: Quantization-aware low-rank adaptation of large language models. *ICLR*, 2024. arXiv:2309.14717.
- Hattie Zhou, Janice Lan, Rosanne Liu, and Jason Yosinski. Deconstructing lottery tickets: Zeros, signs, and the supermask. *NeurIPS*, 2019. arXiv:1905.01067.